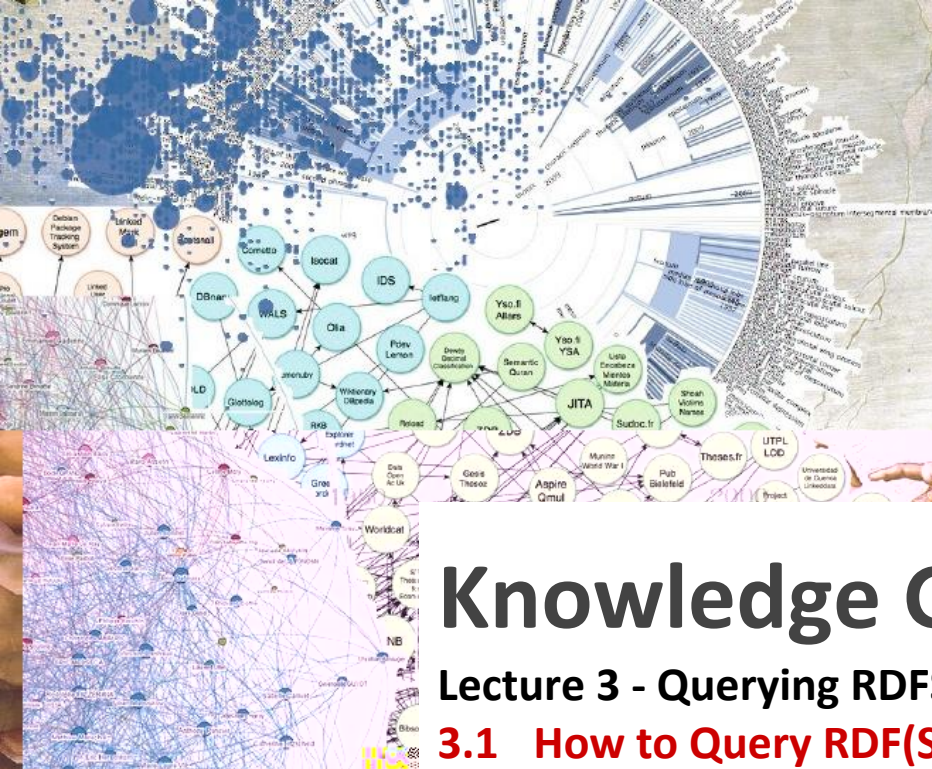




Knowledge Graphs

Lecture 3 - Querying RDFS with SPARQL

3.1 How to Query RDF(S)

Prof. Dr. Harald Sack & Dr. Mehwish Alam

FIZ Karlsruhe - Leibniz Institute for Information Infrastructure

AIFB - Karlsruhe Institute of Technology

Autumn 2020



FIZ Karlsruhe

Leibniz-Institut für Informationsinfrastruktur

Knowledge Graphs

Lecture 3: Querying RDF(S) with SPARQL

3.1 How to Query RDF(S)

Excursion 2: DBpedia Knowledge Graph

Excursion 3: Wikidata Knowledge Graph

3.2 Complex Queries with SPARQL

3.3 More Complex SPARQL Queries

3.4 SPARQL Subqueries and Property Paths

3.5 RDF Databases

3.6 SPARQL is more than a Query Language

The Semantic Web Technology Stack (not a piece of cake...)

Most apps use only a subset of the stack

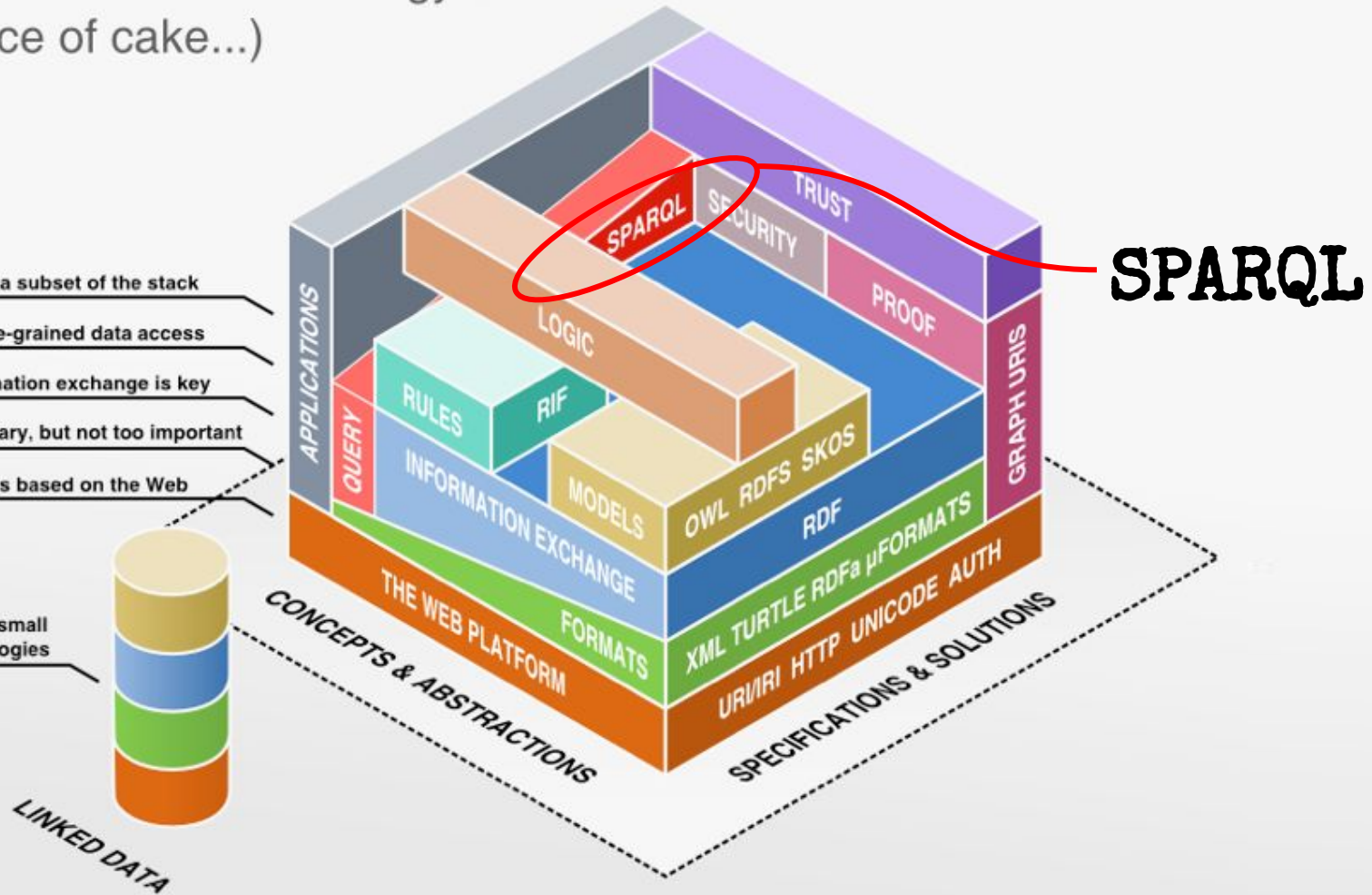
Querying allows fine-grained data access

Standardized information exchange is key

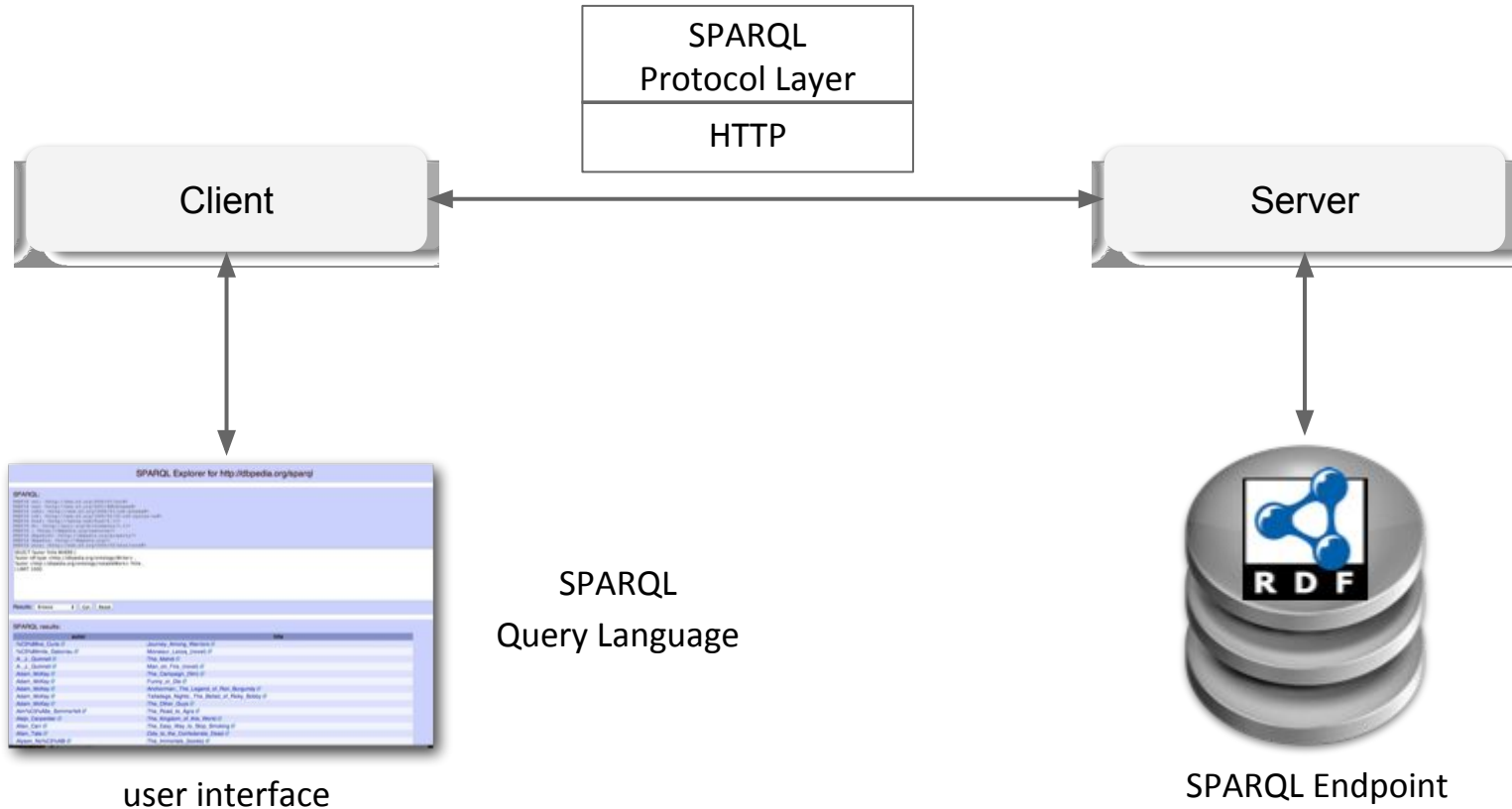
Formats are necessary, but not too important

The Semantic Web is based on the Web

Linked Data uses a small selection of technologies



SPARQL - A Query Language for RDF(S)

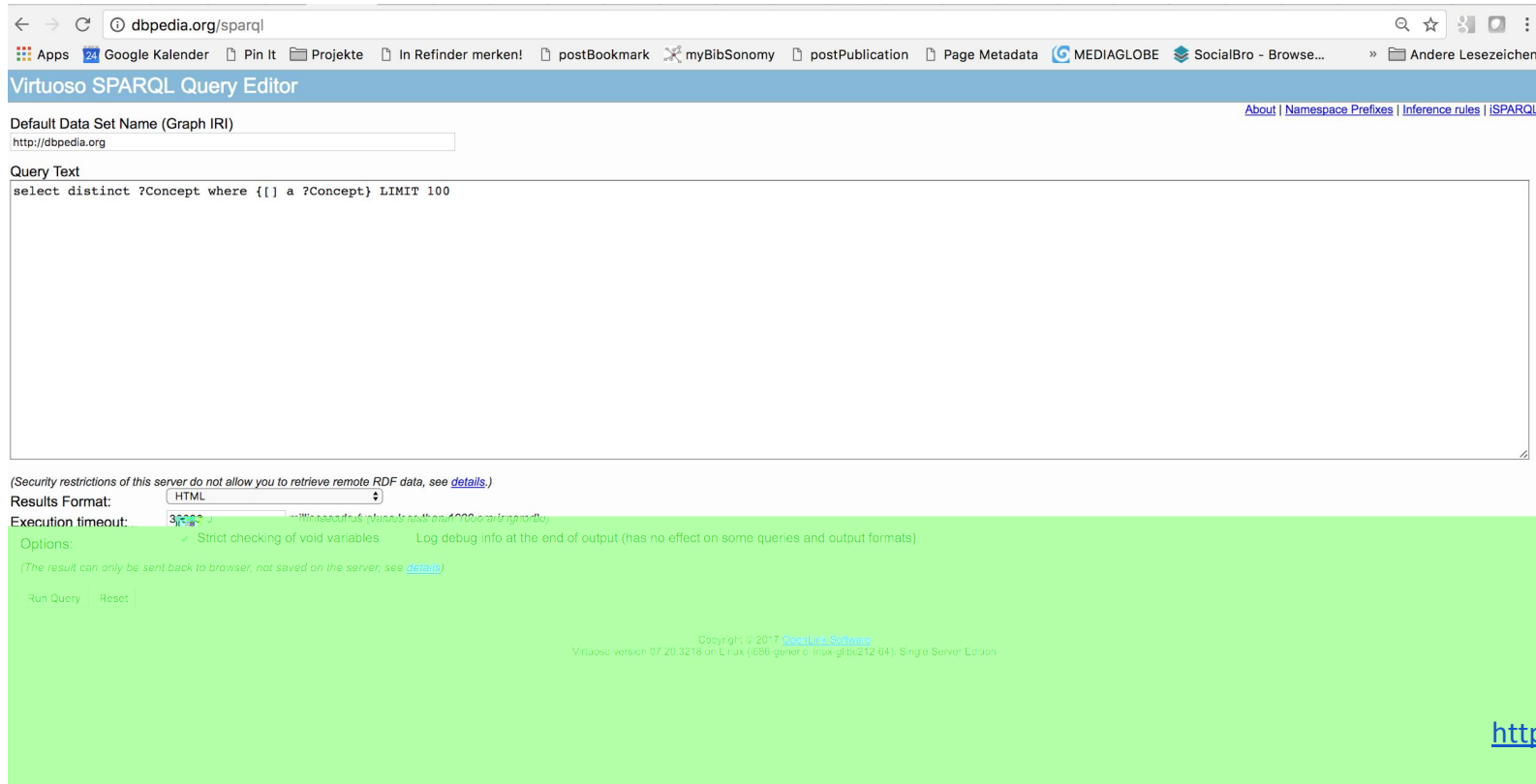


SPARQL - A Query Language for RDF(S)

- **SPARQL Protocol and RDF Query Language** is
 - a **Query Language** for RDF graph traversal
(*SPARQL Query Language Specification*)
 - a **Protocol Layer**, to use SPARQL via http
(*SPARQL Protocol for RDF Specification*)
 - an **XML Output Format Specification** for SPARQL queries
(*SPARQL Query XML Results Format*)
 - W3C Standard (SPARQL 1.1, Mar 2013)
 - inspired by SQL

3. Querying RDF(S) with SPARQL / 3.1 How to Query RDF(S)

SPARQL - Endpoint Example



The screenshot shows the Virtuoso SPARQL Query Editor interface in a web browser. The address bar shows the URL `dbpedia.org/sparql`. The browser's tab bar includes links to Apps, Google Kalender, Pin it, Projekte, In Refinder merken!, postBookmark, myBibSonomy, postPublication, Page Metadata, MEDIAGLOBE, SocialBro - Browse..., and Andere Lesezeichen.

The page title is "Virtuoso SPARQL Query Editor". Below the title, there are links for [About](#), [Namespace Prefixes](#), [Inference rules](#), and [SPARQL](#).

The "Default Data Set Name (Graph IRI)" field contains the URL `http://dbpedia.org`.

The "Query Text" area contains the following SPARQL query:

```
select distinct ?Concept where {[ ] a ?Concept} LIMIT 100
```

Below the query text, there is a note: "(Security restrictions of this server do not allow you to retrieve remote RDF data, see [details](#).)".

The "Results Format" dropdown menu is set to "HTML".

The "Execution timeout" is set to "30s".

The "Options" section includes:

- ☒ Strict checking of void variables
- ☐ Log debug info at the end of output (has no effect on some queries and output formats)

A note below the options states: "(The result can only be sent back to browser, not saved on the server, see [details](#))".

At the bottom of the interface, there are buttons for "Run Query" and "Reset".

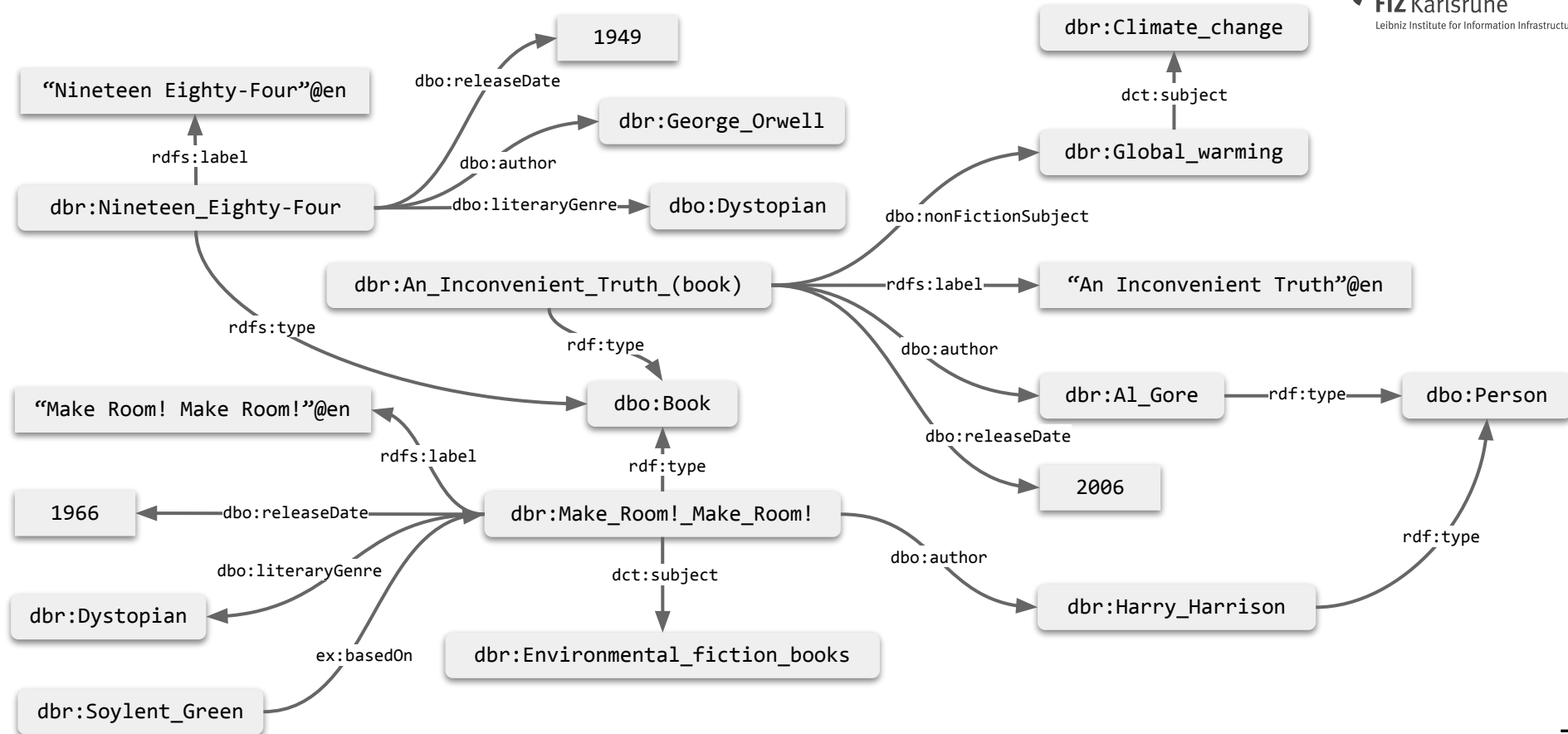
The footer of the interface displays the Virtuoso version: "Virtuoso version 07.20.3218 on Linux (686 generic linux-glibc212.64). Single Server Edition".

The URL <http://dbpedia.org/sparql> is displayed in the bottom right corner of the interface.



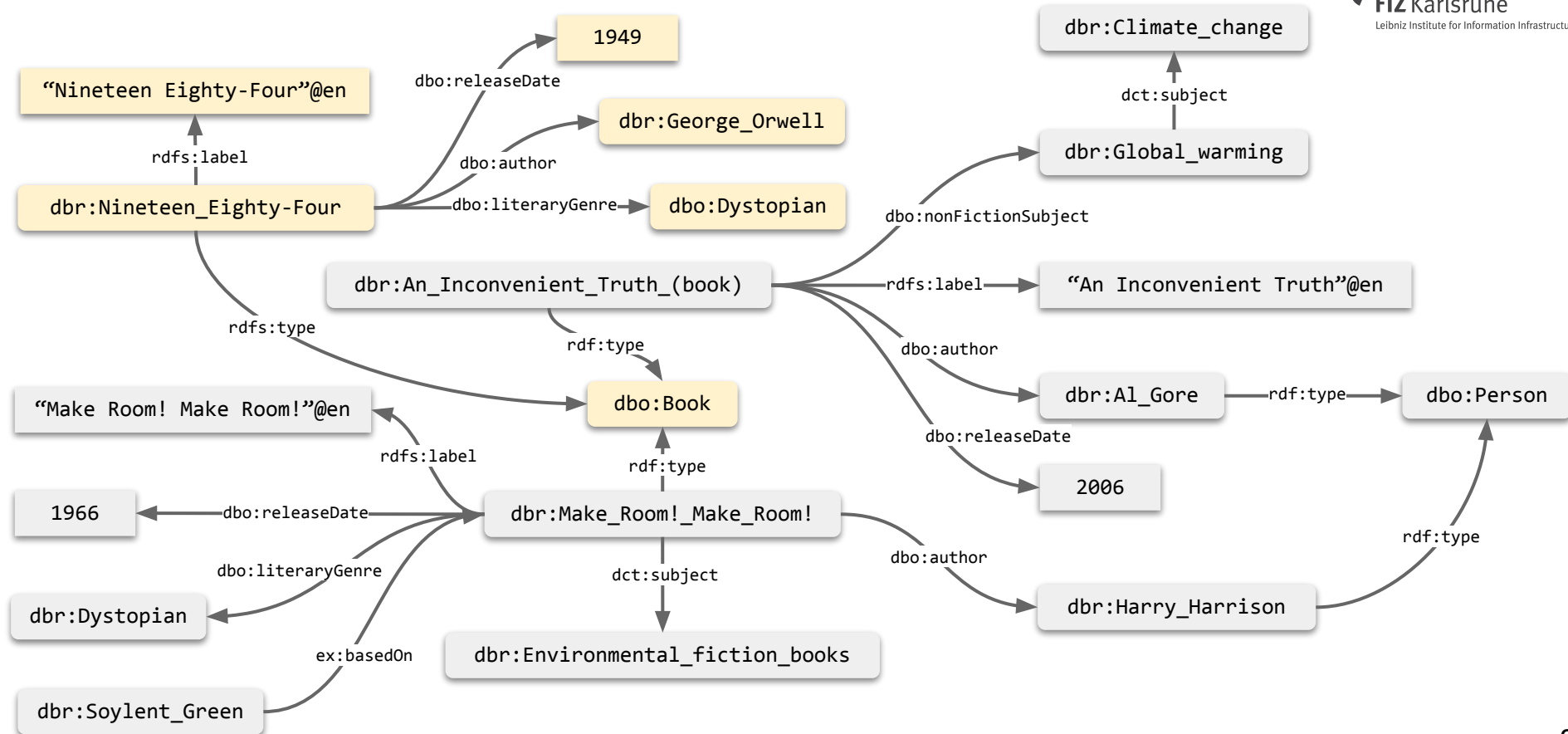
3. Querying RDF(S) with SPARQL / 3.1 How to Query RDF(S)

Querying an RDF-based Knowledge Graph

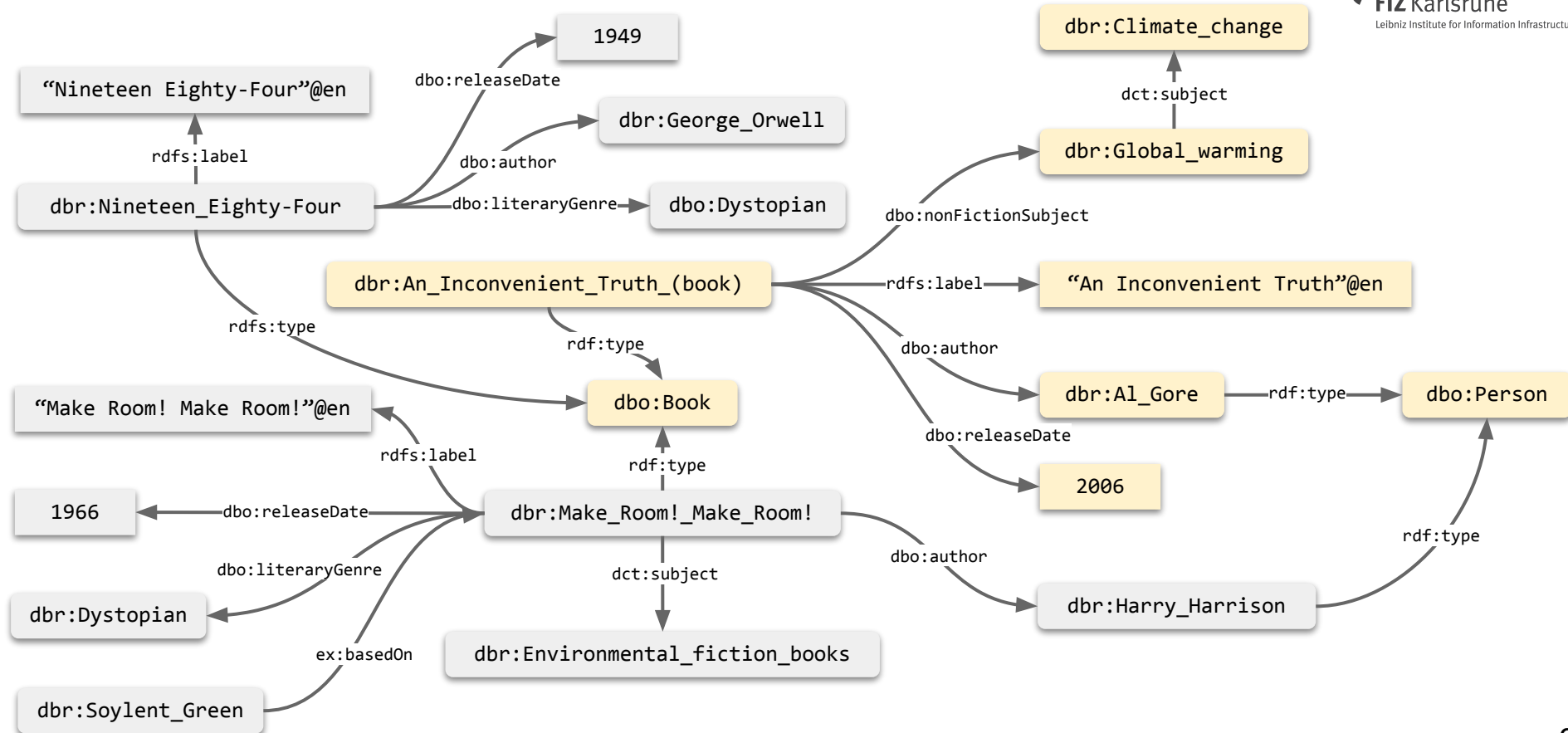


3. Querying RDF(S) with SPARQL / 3.1 How to Query RDF(S)

Querying an RDF-based Knowledge Graph

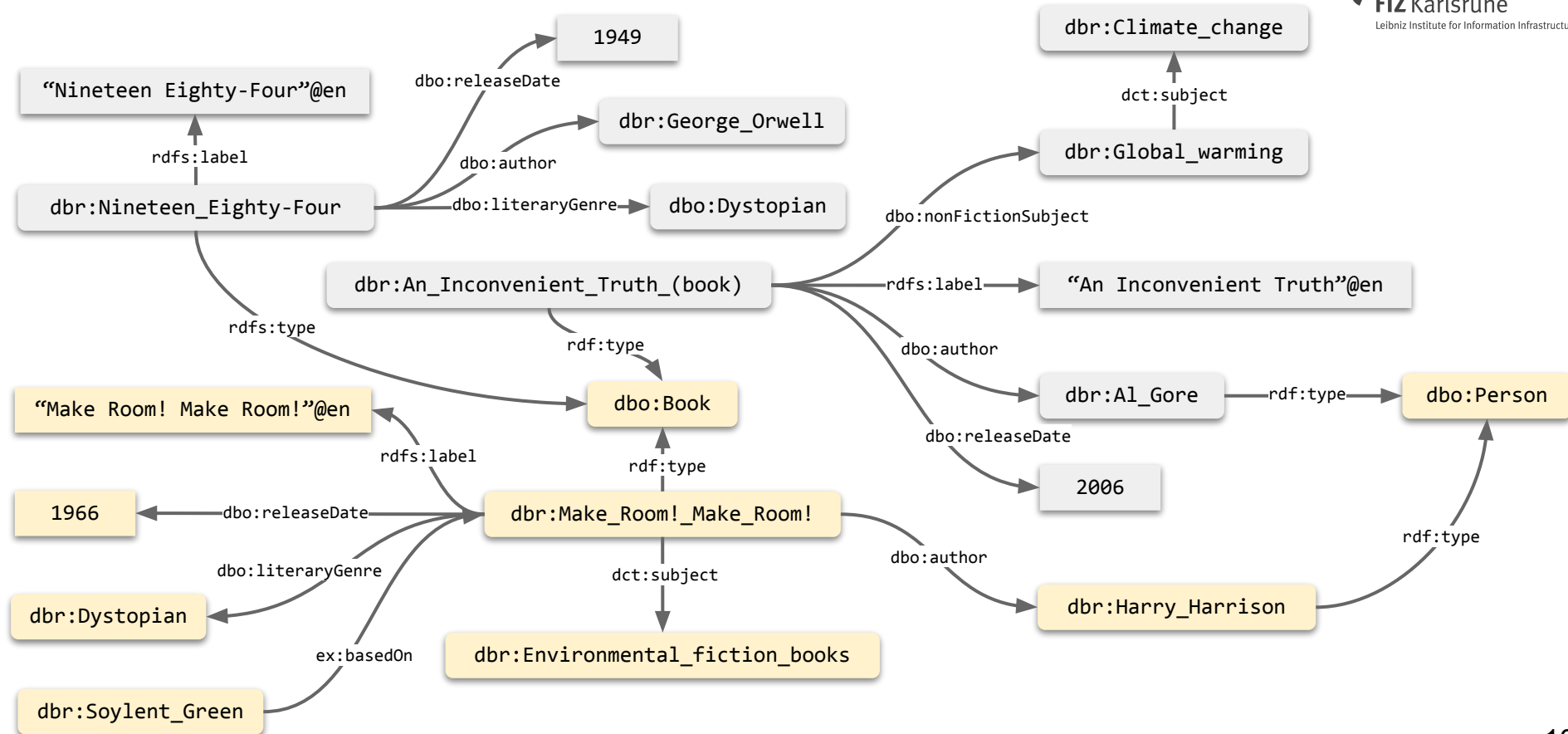


Querying an RDF-based Knowledge Graph



3. Querying RDF(S) with SPARQL / 3.1 How to Query RDF(S)

Querying an RDF-based Knowledge Graph



For Queries we need Variables

- SPARQL **variables** are bound to RDF terms
 - e.g. **?title, ?author, ?date**
- In the same way as in SQL,
a **Query for variables** is performed via **SELECT statement**
 - e.g. **SELECT ?title ?author ?date**
- A SELECT statement returns query results as a **table**

SPARQL Query

SPARQL Result

?title	?author	?date
Nineteen Eighty-Four	George Orwell	1948
An Inconvenient Truth	Al Gore	2006
Make Room! Make Room!	Harry Harrison	1966

SPARQL Graph Pattern Matching

- SPARQL is based on
(1) **RDF Turtle serialization** and (2) **basic graph pattern matching**.
- A **Graph Pattern (Triple Pattern)** is a RDF Triple that contains variables at any arbitrary place (Subject, Property, Object).

Graph Pattern (Triple Pattern) = Turtle + Variables

- Example:

Look for *books* and their *authors* (via property *dbo:author*):

?book **dbo:author** **?author** .



variables

SPARQL Graph Pattern Matching

dbr:An_Inconvenient_Truth_(book)

dbr:Make_Room!_Make_Room!

SPARQL Complex Pattern Matching

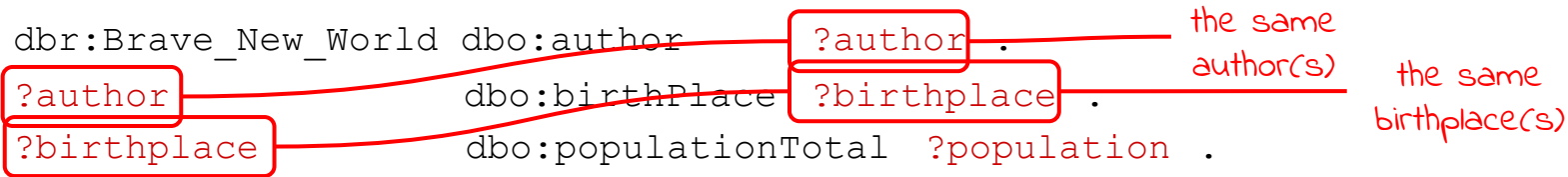
- SPARQL Graph Pattern can be combined to form **complex (conjunctive) queries** for RDF graph traversal.
- *Find books, their authors, and their literary genres:*

the same
book(s)

```
?book dbo:author ?author .  
?book dbo:literaryGenre ?genre .
```

SPARQL Complex Pattern Matching

- SPARQL Graph Pattern can be combined to form **complex (conjunctive) queries** for RDF graph traversal.
- *Given a book URI, find its author(s), the birthplace(s) of its author(s), including the number of population of the birthplace(s):*



SPARQL Complex Pattern Matching

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>
```

specifies namespaces

```
SELECT ?author_name ?title
```

specifies output variables

```
FROM <http://dbpedia.org/>
```

specifies graph to be queried

```
WHERE {
  ?author rdf:type dbo:Writer .
  ?author rdfs:label ?author_name .
  ?author dbo:notableWork ?work .
  ?work rdfs:label ?title .
}
```

*specifies graph pattern
to be matched*

- Example:
search for all
authors and the
titles of their
notable works:



[query SPARQL endpoint](#)

SPARQL Complex Pattern Matching

```
PREFIX :      <http://dbpedia.org/resource/>
PREFIX rdf:   <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo:   <http://dbpedia.org/ontology/>

SELECT ?author_name ?title

FROM <http://dbpedia.org/>

WHERE {
    ?author rdf:type dbo:Writer .
    ?author rdfs:label ?author_name .
    ?author dbo:notableWork ?work .
    ?work rdfs:label ?title .
}

ORDER BY ASC (?author_name)
LIMIT 100
OFFSET 10
```

*solution sequence
modifiers*

- Example:
search for all **authors**
and the **titles** of their
notable works: ordered
by authors in ascending
order and limit the
results to the first 100
results starting the list
at offset 10 position:



[query SPARQL endpoint](#)

SPARQL Filter Constraints

```
PREFIX : <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>

SELECT ?author_name ?title ?pages
FROM <http://dbpedia.org/>
WHERE {
    ?author rdf:type dbo:Writer .
    ?author rdfs:label ?author_name .
    ?author dbo:notableWork ?work .
    ?work dbo:numberOfPages ?pages
    FILTER (?pages > 500) .
    ?work rdfs:label ?title .
} LIMIT 100
```

*specifies constraints
for the result*

- Example:
search for all
authors and the
titles of their notable
works that have
more than 500 pages
and limit the results
to the first 100

- FILTER expressions contain operators and functions

SPARQL Unary Operator Constraints

Operator	Type(A)	Result Type
!A	xsd:boolean	xsd:boolean
+A	numeric	numeric
-A	numeric	numeric
BOUND (A)	variable	xsd:boolean
isURI (A)	RDF term	xsd:boolean
isBLANK (A)	RDF term	xsd:boolean
isLITERAL (A)	RDF Term	xsd:boolean
STR (A)	literal/URL	simple literal
LANG (A)	literal	simple literal
DATATYPE (A)	literal	URI

SPARQL Filter Constraints

```
PREFIX : <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dct: <http://purl.org/dc/terms/>
PREFIX dbc: <http://dbpedia.org/resource/Category:>

SELECT ?author_name ?title
FROM <http://dbpedia.org/>
WHERE {
    ?author rdf:type dbo:Writer .
    ?author rdfs:label ?author_name
    FILTER (LANG(?author_name)="en") .
    ?work dbo:author ?author .
    ?work rdfs:label ?title .
    FILTER (LANG(?title)="en")
    ?work dct:subject dbc:Environmental_fiction_books .
} LIMIT 100
```

- Example:
Search for **authors**
and their **books**, filter
results for **English**
labels and
Environmental fiction
books and limit the
results to the **first**
100





Excursion 2: DBpedia Knowledge Graph

Picture References:

- [1] Benjamin Nowack, *The Semantic Web - Not a Piece of cake...*, at [bnode.org](http://bnode.org/blog/2009/07/08/the-semantic-web-not-a-piece-of-cake), 2009-07-08 , [CC BY 3.0]
<http://bnode.org/blog/2009/07/08/the-semantic-web-not-a-piece-of-cake>
- [2] DBpedia logo, wiki.dbpedia.org, DBpedia Team [Public Domain]
<https://commons.wikimedia.org/wiki/File:DBpediaLogo.svg>
- [3] The Linked Open Data Cloud, lod-cloud.net, [CC-BY]
<https://lod-cloud.net/clouds/lod-cloud.svg>